

TXSDK WiFi 开发指南



保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

责任与版权

责任限制

由于产品版本升级或者其他原因，本文档会不定期更新。除非另行约定，泰芯半导体有限公司对本文档所有内容不提供任何担保或授权。

客户应在遵守法律、法规和安全要求的前提下进行产品设计，并做充分验证。泰芯半导体有限公司对应用帮助或客户产品设计不承担任何义务。客户应对其使用泰芯半导体有限公司的产品和应用自行负责。

在适用法律允许的范围内，泰芯半导体有限公司在任何情况下，都不对因使用本文档相关内容及本文档描述的产品而产生的损失和损害进行超过购买支付价款的赔偿（除在涉及人身伤害的情况中根据适用的法律规定的损害赔偿外）。

版权申明

泰芯半导体有限公司保留随时修改本文档中任何信息的权利，无需提前通知且不承担任何责任。

未经泰芯半导体有限公司书面同意，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。除非获得相关权利人的许可，否则，任何人不能以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让、分许可等侵犯本文档描述的享有版权的软件版权的行为，但是适用法禁止此类限制的除外。



珠海泰芯半导体有限公司
Zhuhai TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港
11 栋 3 楼

版权所有 侵权必究

Copyright © 2023 by TaiXin All Rights Reserved

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

修订记录

日期	版本	描 述	修订人
2024-05-07	V1.0		TX

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

目录

TXSDK WiFi 开发指南 1

1. 概述1

2. WiFi 工作模式2

 2.1. AP 模式 2

 2.2. STA 模式 2

 2.3. 中继模式 2

 2.4. P2P 模式 3

 2.5. WNBAP 模式 3

 2.6. WNBSTA 模式 3

 2.7. TXMesh 模式 3

3. WiFi 协议栈初始化 4

 3.1. AP 模式初始化 5

 3.2. STA 模式初始化 6

 3.3. 中继模式初始化 7

 3.4. P2P 模式初始化 8

 3.5. WNBAP 模式初始化 9

 3.6. WNBSTA 模式初始化 9

 3.7. Mesh 模式初始化 10

4. WiFi 协议栈 Event 机制 11

 4.1. WiFi 协议栈 Event 列表 12

 4.1.1. IEEE80211_EVENT_PAIR_START 12

 4.1.2. IEEE80211_EVENT_PAIR_SUCCESS 12

 4.1.3. IEEE80211_EVENT_PAIR_DONE 12

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

4.1.4.	IEEE80211_EVENT_SCAN_START	13
4.1.5.	IEEE80211_EVENT_SCAN_DONE	13
4.1.6.	IEEE80211_EVENT_CONNECT_START	13
4.1.7.	IEEE80211_EVENT_CONNECTED	14
4.1.8.	IEEE80211_EVENT_CONNECT_FAIL	14
4.1.9.	IEEE80211_EVENT_DISCONNECTED	14
4.1.10.	IEEE80211_EVENT_RX_FRAME	15
4.1.11.	IEEE80211_EVENT_RSSI	15
4.1.12.	IEEE80211_EVENT_NEW_BSS	15
4.1.13.	IEEE80211_EVENT_UPDATE_BSS	16
4.1.14.	IEEE80211_EVENT_STA_PS_START	16
4.1.15.	IEEE80211_EVENT_STA_PS_END	16
4.1.16.	IEEE80211_EVENT_INTERFACE_ENABLE	17
4.1.17.	IEEE80211_EVENT_INTERFACE_DISABLE	17
4.1.18.	IEEE80211_EVENT_ADD_CUSTOMER_IE	17
4.1.19.	IEEE80211_EVENT_MAC_CHANGE	18
4.1.20.	IEEE80211_EVENT_EVM	18
4.1.21.	IEEE80211_EVENT_PRE_AUTH	18
4.1.22.	IEEE80211_EVENT_PRE_ASSOC	19
4.1.23.	IEEE80211_EVENT_UNPAIR_SUCCESS	19
4.1.24.	IEEE80211_EVENT_TX_BITRATE	19
4.1.25.	IEEE80211_EVENT_TX_FRAME	20
4.1.26.	IEEE80211_EVENT_RX_CUSTMGMT	20
4.1.27.	IEEE80211_EVENT_WRONG_KEY	21

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

4.1.28.	IEEE80211_EVENT_CHANNEL_CHANGE	21
4.1.29.	IEEE80211_EVENT_TX_STATUS	21
4.1.30.	IEEE80211_EVENT_PAIR_NGO	22
4.2.	WiFi 协议栈 sysevt 机制	23
5.	WiFi 协议栈 Hook 机制	25
6.	WiFi 协议栈 API	27
6.1.	初始化 API	27
6.1.1.	ieee80211_init	27
6.1.2.	ieee80211_support_twx830x	27
6.1.3.	ieee80211_support_twx80x	28
6.1.4.	ieee80211_support_twx81x	28
6.1.5.	ieee80211_iface_create_ap	28
6.1.6.	ieee80211_iface_create_sta	28
6.1.7.	ieee80211_iface_create_wnbap	29
6.1.8.	ieee80211_iface_create_wnbsta	29
6.1.9.	ieee80211_iface_create_p2pdev	30
6.1.10.	ieee80211_iface_start	30
6.1.11.	ieee80211_iface_stop	30
6.1.12.	ieee80211_deliver_init	31
6.1.13.	ieee80211_conf_stabr_table	31
6.1.14.	wpa_passphrase	31
6.2.	基础配置 API	32
6.2.1.	ieee80211_conf_set_channel	32
6.2.2.	ieee80211_conf_set_ssid	32

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

6.2.3.	ieee80211_conf_set_keymgmt	32
6.2.4.	ieee80211_conf_set_psk	33
6.2.5.	ieee80211_conf_set_passwd	33
6.2.6.	ieee80211_conf_set_beacon_int	33
6.3.	高级配置 API	34
6.3.1.	ieee80211_conf_set_bssbw	34
6.3.2.	ieee80211_conf_set_dtim_int	34
6.3.3.	ieee80211_conf_set_bss_max_idle	34
6.3.4.	ieee80211_conf_set_bssid	35
6.3.5.	ieee80211_conf_set_mac	35
6.3.6.	ieee80211_conf_set_aphide	35
6.3.7.	ieee80211_conf_set_hwmode	36
6.3.8.	ieee80211_expand_mlme_size	36
6.3.9.	ieee80211_conf_set_wpa_cipher	37
6.3.10.	ieee80211_conf_set_isolate	37
6.3.11.	ieee80211_conf_set_chanlist	37
6.3.12.	ieee80211_conf_set_wpa_group_rekey	38
6.3.13.	ieee80211_conf_set_dupfilter	38
6.3.14.	ieee80211_conf_set_conn_timeout	38
6.3.15.	ieee80211_conf_set_aplost_time	39
6.3.16.	ieee80211_conf_set_wmm_enable	39
6.3.17.	ieee80211_conf_set_wmm_param	40
6.3.18.	ieee80211_conf_set_use4addr	40
6.3.19.	ieee80211_conf_set_psdata_cnt	41

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

6.3.20.	ieee80211_conf_set_datatag	41
6.3.21.	ieee80211_conf_set_signal_threshold	41
6.4.	一键配对 API	42
6.4.1.	ieee80211_pair_enable	42
6.4.2.	ieee80211_pairing	42
6.4.3.	ieee80211_unpair	43
6.4.4.	ieee80211_conf_set_pair_ngo	43
6.5.	连接扫描 API	44
6.5.1.	ieee80211_scan	44
6.5.2.	ieee80211_get_bsslist	44
6.5.3.	ieee80211_cleanup_bsslist	45
6.5.4.	ieee80211_conf_set_auto_assoc	45
6.5.5.	ieee80211_start_connect	45
6.5.6.	ieee80211_disassoc	46
6.5.7.	ieee80211_disassoc_all	46
6.5.8.	ieee80211_bss_add_manualAP	46
6.5.9.	ieee80211_conf_set_scan_max	47
6.6.	数据收发 API	47
6.6.1.	ieee80211_scatter_tx	47
6.6.2.	ieee80211_tx	48
6.6.3.	ieee80211_input	48
6.6.4.	ieee80211_tx_mgmt	48
6.6.5.	ieee80211_tx_custgmt	49
6.7.	Hook 和 Event API	49

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

6.7.1.	ieee80211_register_pkthook	49
6.7.2.	ieee80211_event_cb	50
6.8.	低功耗 API	50
6.8.1.	ieee80211_conf_wakeup_sta	50
6.8.2.	ieee80211_conf_get_wkreason	51
6.9.	信息获取 API	51
6.9.1.	ieee80211_status	51
6.9.2.	ieee80211_conf_get_psk	52
6.9.3.	ieee80211_conf_get_keymgmt	52
6.9.4.	ieee80211_conf_get_ssid	52
6.9.5.	ieee80211_get_stalist	52
6.9.6.	ieee80211_conf_get_bssbw	53
6.9.7.	ieee80211_conf_get_bssid	53
6.9.8.	ieee80211_conf_get_channel	53
6.9.9.	ieee80211_conf_get_mac	54
6.9.10.	ieee80211_conf_get_connstate	54
6.9.11.	ieee80211_conf_get_stainfo	54
6.9.12.	ieee80211_conf_get_stalist	55
6.9.13.	ieee80211_conf_get_stacnt	55
6.9.14.	ieee80211_conf_get_reason_code	55
6.9.15.	ieee80211_conf_get_status_code	56
6.9.16.	ieee80211_chan_center_freq	56
6.9.17.	ieee80211_bss_get_wpadata	56
6.10.	WiFi 漫游 API	57

保密等级	A	TXSDK WiFi 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

6.10.1. 标准协议漫游 ·····

57

6.10.2. 私有协议漫游 ·····

58

7. 示例代码

60

7.1. 开启 AP 热点 ·····

60

7.2. 开启 STA 连接 AP ·····

60

1. 概述

TXSDK 是泰芯半导体发布的 WiFi 系列芯片开发 SDK。本文档是 TXSDK 的 WiFi 功能开发指南，介绍了 SDK 的 WiFi 协议栈所支持的功能以及使用方法，适用于所有的泰芯 WiFi 芯片 SDK。

SDK 的 WiFi 协议栈同时支持标准 WiFi 协议和泰芯私有 WiFi 协议，支持标准协议和私有协议互通。可以根据产品实际需求，选择标准协议或私有协议。

SDK 的 WiFi 协议栈的特性综述如下：

1. 工作模式：AP, STA, P2P, APSTA, WNBAP, WNBSTA, TXMESH
2. 安全模式：OPEN, WPA2-PSK, WPA3-PSK
3. 协议标准：WiFi 标准协议 (11BGN, 11AX, 11AH) ， 泰芯私有 WiFi 协议
4. 一键配对：支持 AP, STA 一键配对组网，简化组网参数设置。
5. 快速漫游：支持 STA 在多个 AP 之间快速漫游。
6. 休眠唤醒：支持 STA 进入低功耗状态，支持休眠唤醒。
7. 无线桥接：支持 WiFi 接口与其他网络接口进行桥接，例如网口。
8. 动态裁剪：未启用的功能代码，在程序链接时会自动移除，减少代码体积。
9. 二次开发：SDK 的 WiFi 协议栈设计了 Event 和 Hook 机制，支持应用代码参与控制协议栈的运行，以满足应用方案对 WiFi 功能进行管理控制的需求（详见第 4, 5 章节）。

2. WiFi 工作模式

SDK 的 WiFi 协议栈支持多种工作模式，可以根据产品方案需求选择不同的工作模式。

2.1. AP 模式

SDK 支持 WiFi AP 模式，提供 WiFi 热点服务。支持 4 地址模式，支持多 STA 接入，支持 STA 进入低功耗状态和休眠唤醒。

该 AP 模式是指标准协议的 AP 功能，例如 TXW81x 芯片可支持常见的手机，无线网卡接入，TXW8301 芯片可支持 802.11ah STA 设备接入。

2.2. STA 模式

SDK 支持 WiFi STA 模式，作为 WiFi 设备接入到其他 WiFi AP。STA 模式支持低功耗功能，支持 4 地址模式桥接和 3 地址模式桥接。

该 STA 模式是指标准协议的 STA 功能，例如 TXW81x 芯片可以连接常见的 WiFi 路由器，TXW8301 芯片可以连接 802.11ah 路由器。

2.3. 中继模式

SDK 支持 WiFi 中继模式，也就是 APSTA 模式。中继模式是 AP 功能和 STA 功能的组合体，可实现无线中继功能。

中继模式时 WiFi 协议栈同时启用 AP 接口和 STA 接口，STA 用于连接上一级 AP，AP 接口则用于提供 AP 热点，接受其他 STA 接入。

中继模式支持自动中继功能，协议栈自动搜寻周围的 AP 热点，选择最优 AP 进行连接，可实现多设备自动组网，中继转发。

2.4. P2P 模式

SDK 支持 WiFi p2p 模式，也就是 WiFi direct 功能。可支持开发 WiFi p2p 类型的应用方案。

2.5. WNBAP 模式

SDK 支持 WNBAP 模式，该模式是泰芯私有 WiFi 协议的 AP 模式，可兼容 v1.x SDK 固件。泰芯私有协议可支持更快速的连接过程，最快 20ms 左右可以完成连接。支持 wnbsta 快速无缝漫游，最快 50ms 左右完成漫游，漫游过程中可实现不丢数据。

注：只支持 WNBSTA 连接。

2.6. WNBSTA 模式

SDK 支持 WNBSTA 模式，该模式是泰芯私有 WiFi 协议的 STA 模式，可以兼容 v1.x SDK 固件。可支持更快速的连接，支持快速漫游。

注：只能连接 WNBAP。

2.7. TXMesh 模式

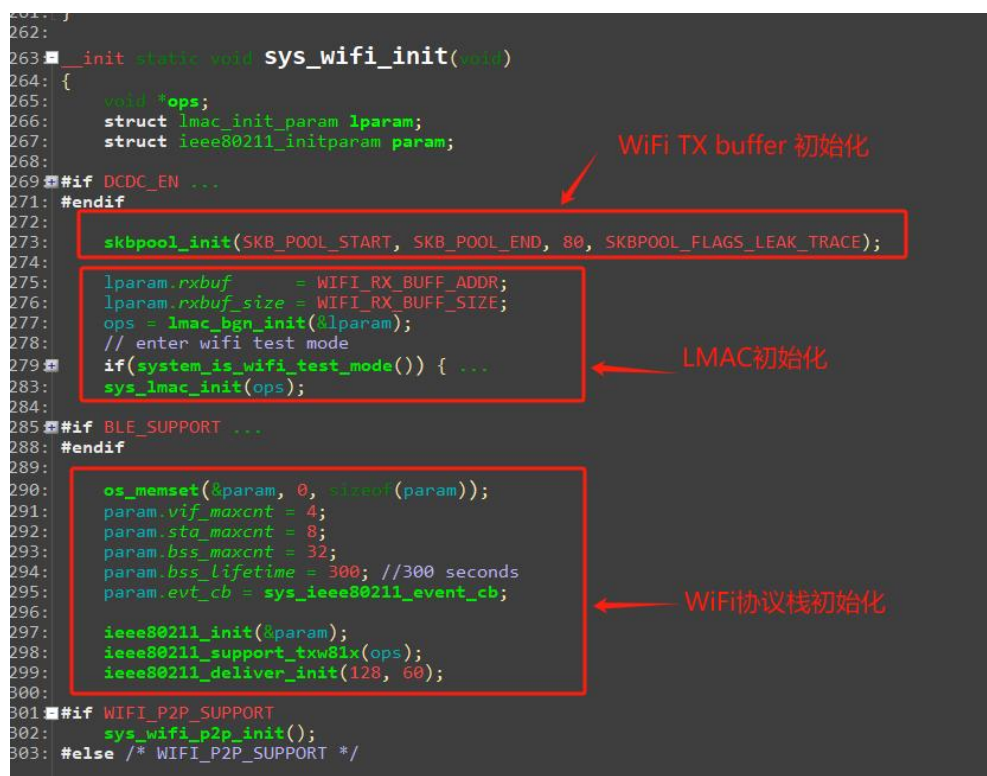
SDK 支持 TXMesh 模式，该模式是泰芯私有 Mesh 协议。支持跳频通信，多节点自动组网转发通信，转发层级无限制，支持 IPv4 和 IPv6。

3. WiFi 协议栈初始化

SDK WiFi 协议栈初始化代码在 main.c 文件中的 sys_wifi_init 函数，包含了 WiFi 协议栈初始化和 WiFi 模式初始化。

WiFi 协议栈所支持的各种工作模式，是以 WiFi 接口形式存在的。所有的 API 都需要接口索引参数，用于表示设置的是哪个 WiFi 接口。

WiFi 协议栈初始化之前需要分配好 TX, RX buffer，在 sys_config.h 文件中定义了 WiFi TX/RX buffer，详细说明请阅读 SDK 入门指南的第 4 章节。



The screenshot shows the `sys_wifi_init` function in `main.c`. The code is annotated with red boxes and arrows indicating the initialization steps:

- WiFi TX buffer 初始化**: Points to the `skbpool_init` call.
- LMAC初始化**: Points to the `sys_lmac_init` call.
- WiFi协议栈初始化**: Points to the `ieee80211_init` and `ieee80211_support_txw81x` calls.

```
262:
263: __init static void sys_wifi_init(void)
264: {
265:     void *ops;
266:     struct lmac_init_param lparam;
267:     struct ieee80211_initparam param;
268:
269: #if DCDC_EN ...
271: #endif
272:
273:     skbpool_init(SKB_POOL_START, SKB_POOL_END, 80, SKBPOOL_FLAGS_LEAK_TRACE);
274:
275:     lparam.rxbuf = WIFI_RX_BUFF_ADDR;
276:     lparam.rxbuf_size = WIFI_RX_BUFF_SIZE;
277:     ops = lmac_bgn_init(&lparam);
278:     // enter wifi test mode
279:     if(system_is_wifi_test_mode()) { ...
283:     sys_lmac_init(ops);
284:
285: #if BLE_SUPPORT ...
288: #endif
289:
290:     os_memset(&param, 0, sizeof(param));
291:     param.vif_maxcnt = 4;
292:     param.sta_maxcnt = 8;
293:     param.bss_maxcnt = 32;
294:     param.bss_lifetime = 300; //300 seconds
295:     param.evt_cb = sys_ieee80211_event_cb;
296:
297:     ieee80211_init(&param);
298:     ieee80211_support_txw81x(ops);
299:     ieee80211_deliver_init(128, 60);
300:
301: #if WIFI_P2P_SUPPORT
302:     sys_wifi_p2p_init();
303: #else /* WIFI_P2P_SUPPORT */
```

如上图所示：

- `skbpool_init` 是 WiFi 协议栈的 TX/RX buffer 初始化。
- `sys_lmac_init` 是 lmac 协议栈初始化。
- `ieee80211_init` 是 WiFi 协议栈初始化，初始化参数说明如下：

```
struct ieee80211_initparam {
    uint8    vif_maxcnt; //支持的接口数量
    uint8    bss_maxcnt; //BSS 列表缓存 AP 信息的最大数量，当数量超过该
值的 1/2 时，AP 信息的生存时间是 5s，5s 之后开始清除缓存信息
    uint8    headroom, tailroom;
    uint16   sta_maxcnt; //支持连接的 sta 最大数量
    uint16   bss_lifetime; //BSS 列表 AP 信息的生存时间，单位：秒
```

```

uint16 stack_size; //WiFi 协议栈的堆栈大小，默认是 2048
uint16 no_rxtask:1, //WiFi 协议栈是否使用独立的 RX Task
      ssid_fuzzy_match:4, //WiFi 协议栈是否开启 SSID 模糊匹配
      rev: 11;
ieee80211_evt_cb evt_cb; //WiFi 协议栈的 event callback
};

```

- ieee80211_support_twx81x 是开启 WiFi 协议栈支持 TXW81x 系列芯片。
- ieee80211_deliver_init 是开启 WiFi 协议栈的转发功能，并指定转发路由表的最大路由信息条数（128 条）和路由信息生存时间（60 秒）。

3.1. AP 模式初始化

AP 模式初始化主要是创建 AP 接口，设置 AP 接口参数，并启动 AP 接口，如下图

所示：

```

__init static void sys_wifi_ap_init(void)
{
    ieee80211_iface_create_ap(WIFI_MODE_AP, IEEE80211_BAND_2GHZ);
    if (sys_cfgs.wifi_mode == WIFI_MODE_AP || sys_cfgs.wifi_mode == WIFI_MODE_APSTA) {
        wificfg_flush(WIFI_MODE_AP);
        ieee80211_iface_start(WIFI_MODE_AP);
    }
}

```

如上图所示：

- ieee80211_iface_create_ap API 是创建 AP 接口，需要指定接口的索引 ID 和 WiFi Band 类型。
 - IEEE80211_BAND_2GHZ：2.4G 类型 WiFi
 - IEEE80211_BAND_5GHZ：5.8G 类型 WiFi
 - IEEE80211_BAND_60GHZ：60G 类型 WiFi
- wificfg_flush 函数是将系统参数刷写到 WiFi 协议栈，并指定接口索引 WIFI_MODE_AP
- ieee80211_iface_start 函数是启动 WiFi 协议栈的功能接口。
- 停止协议栈功能接口的 API 是 ieee80211_iface_stop 函数。

3.2. STA 模式初始化

STA 模式初始化主要是创建 STA 接口，设置 STA 接口参数，并启动 STA 接口，如下图所示：

```
_init static void sys_wifi_sta_init(void)
{
    ieee80211_iface_create_sta(WIFI_MODE_STA, IEEE80211_BAND_2GHZ);
    ieee80211_conf_set_use4addr(WIFI_MODE_STA, 1);
    ieee80211_conf_stabr_table(WIFI_MODE_STA, 128, 10 * 60);
    if (sys_cfgs.wifi_mode == WIFI_MODE_STA || sys_cfgs.wifi_mode == WIFI_MODE_APSTA) {
        wificfg_flush(WIFI_MODE_STA);
        ieee80211_iface_start(WIFI_MODE_STA);
    }
}
```

如上图所示：

- ieee80211_iface_create_sta API 是创建 STA 接口，需要指定接口的索引 ID 和 WiFi Band 类型。
- ieee80211_conf_set_use4addr 是设置支持 sta 接口的 4 地址模式，可用于 4 地址桥接功能。
- ieee80211_conf_stabr_table 设置开启 sta 接口的 3 地址桥接功能，并指定路由表信息最大数量和生存时间。桥接功能可根据实际产品需要选择关闭，或开启 3 地址桥接，或开启 4 地址桥接。
 - 3 地址桥接和 4 地址桥接的功能区别：
 - 3 地址桥接：
 - 优点：使用 3 地址桥接时，不需要对路由器做任何设置。
 - 缺点：有防火墙管控的网络，可能会造成桥接后的设备不能访问外部网络。
 - 4 地址桥接：
 - 优点：使用 4 地址桥接时，可以不受防火墙的影响。
 - 缺点：需要设置路由器支持 4 地址。可能部分路由器不支持，或者需要修改参数才能支持。

SDK 的无线网桥方案的 demo 代码针对 3 地址和 4 地址桥接功能做了自适应适配，

自动检查当前网络环境是适用 3 地址模式，还是 4 地址模式。

3.3. 中继模式初始化

中继模式是 AP 功能和 STA 功能的组合体，所以中继模式的初始化需要同时初始化 AP 模式和 STA 模式。WIFI_MODE_APSTA 宏定义值并不是协议栈的接口索引。创建接口的索引还是 WIFI_MODE_AP 和 WIFI_MODE_STA。

开启中继功能时参数设置有所差别，其中 ssid/psk 用于 STA 接口连接上一级 AP，r_ssid/r_psk 则用于 AP 接口，提供 AP 热点服务。如下图所示：

```
92:
93: if(ifidx == WIFI_MODE_AP){
94:     if(sys_cfgs.channel==0 && sys_status.channel){
95:         ieee80211_conf_set_channel(ifidx, sys_status.channel);
96:     }else{
97:         ieee80211_conf_set_channel(ifidx, sys_cfgs.channel);
98:     }
99: }
100: ieee80211_conf_set_beacon_int(ifidx, sys_cfgs.beacon_int);
101: ieee80211_conf_set_dtim_int(ifidx, sys_cfgs.dtim_period);
102: ieee80211_conf_set_bss_max_idle(ifidx, sys_cfgs.bss_max_idle);
103: ieee80211_conf_set_apbss(ifidx, sys_cfgs.ap_bss);
104: ieee80211_conf_set_bssbw(ifidx, sys_cfgs.bss_bw);
105: if(ifidx == WIFI_MODE_AP && sys_cfgs.wifi_mode == WIFI_MODE_APSTA && sys_cfgs.cfg_init){
106:     ieee80211_conf_set_ssid(ifidx, sys_cfgs.r_ssid);
107:     ieee80211_conf_set_keymgmt(ifidx, sys_cfgs.r_key_mgmt);
108:     ieee80211_conf_set_psk(ifidx, sys_cfgs.r_psk);
109: }else{
110:     ieee80211_conf_set_ssid(ifidx, sys_cfgs.ssid);
111:     ieee80211_conf_set_keymgmt(ifidx, sys_cfgs.key_mgmt);
112:     ieee80211_conf_set_psk(ifidx, sys_cfgs.psk);
113: }
114: return 0;
115: } // end wifi_cfg_flush
116:
117: void netcfg_flush(void)
118: {
119:     in_addr_t ipaddr, netmask, gw;
```

使用 ieee80211_conf_set_relay_mode API 开启自动中继功能。实际上不执行该 API，开启中继功能的 WiFi 协议栈也会自动搜索连接周围的 AP，实现自动中继的效果。但是在链路择优，避免链路回环方面可能会有问题。所以当需要多设备中继组网通信时请开启自动中继功能。

3.4. P2P 模式初始化

WiFi P2P 功能是 2 个 WiFi 设备通过协议协商选择一个做 GO (AP), 另一个做 GC (STA), 然后建立 WiFi 连接。所以 P2P 模式的初始化涉及到 AP 接口, STA 接口, P2P 接口的初始化。如下图所示:

```
#if WIFI_P2P_SUPPORT
__init static void sys_wifi_p2p_init(void)
{
    ieee80211_iface_create_ap(WIFI_MODE_AP, IEEE80211_BAND_2GHZ);
    //wificfg_flush(WIFI_MODE_AP);
    ieee80211_conf_set_mac(WIFI_MODE_AP, sys_cfgs.mac);
    ieee80211_iface_create_sta(WIFI_MODE_STA, IEEE80211_BAND_2GHZ);
    ieee80211_conf_stabr_table(WIFI_MODE_STA, 128, 10 * 60);
    //wificfg_flush(WIFI_MODE_STA);
    ieee80211_conf_set_mac(WIFI_MODE_STA, sys_cfgs.mac);
    ieee80211_iface_create_p2pdev(WIFI_MODE_P2P, IEEE80211_BAND_2GHZ, WIFI_MODE_AP, WIFI_MODE_STA);
    //wificfg_flush(WIFI_MODE_P2P);
    ieee80211_conf_set_mac(WIFI_MODE_P2P, sys_cfgs.mac);
    ieee80211_conf_set_ssid(WIFI_MODE_P2P, sys_cfgs.ssid);
    ieee80211_conf_set_keymgmt(WIFI_MODE_P2P, sys_cfgs.key_mgmt);
    ieee80211_conf_set_psk(WIFI_MODE_P2P, sys_cfgs.psk);
    ieee80211_iface_start(WIFI_MODE_P2P);
}
#endif
```

开启 p2p 功能后, 通过手机可搜索发现 p2p 设备, 并进行连接。可以根据应用需要, 控制 WiFi 设备是做 GO, 还是 GC。

3.5. WNBAP 模式初始化

WNBAP 模式是泰芯私有协议的 AP 模式，初始化工作主要是创建 WNBAP 接口，并设置参数。如下图所示：

```
__init static void sys_wifi_wnbap_init(void)
{
    ieee80211_iface_create_wnbap(WIFI_MODE_WNBAP, IEEE80211_BAND_S1GHZ);
    ieee80211_pair_enable(WIFI_MODE_WNBAP, WIFI_PAIR_MAGIC);
    wificfg_flush(WIFI_MODE_WNBAP);
}
```

参数设置,使用方式 和 AP 模式一样。示例代码中的 wnbap 接口开启了一键配对功能。

私有协议的 AP 接口和 STA 接口，仅仅是初始化 API 不同，其他参数设置 API 和使用方式与标准协议接口没有区别。

3.6. WNBSTA 模式初始化

WNBAP 模式是泰芯私有协议的 STA 模式，初始化工作主要是创建 WNBSTA 接口，并设置参数。如下图所示：

```
__init static void sys_wifi_wnbsta_init(void)
{
    ieee80211_iface_create_wnbsta(WIFI_MODE_WNBSTA, IEEE80211_BAND_S1GHZ);
    ieee80211_pair_enable(WIFI_MODE_WNBSTA, WIFI_PAIR_MAGIC);
    ieee80211_wnb_roam_enable(WIFI_MODE_WNBSTA);
    wificfg_flush(WIFI_MODE_WNBSTA);
}
```

参数设置和使用方式，与 STA 模式一样。示例代码的 wnbsta 接口开启了一键配对和快速漫游功能。

3.7. Mesh 模式初始化

SDK 的 WiFi 协议栈支持 Mesh 功能，并支持跳频通信。协议栈实现的 Mesh 功能是树状网络，支持多节点多跳通信，支持 IPv4 和 IPv6。Mesh 功能适用于多设备自组网通信，例如智能电表。

4. WiFi 协议栈 Event 机制

SDK WiFi 协议栈设计了 event 机制。协议栈运行过程中在不同的阶段会产生不同的 event，通过这些 event 应用程序可以参与控制 WiFi 协议栈的运行。

协议栈的 event 处理函数在初始化时通过 [evt_cb](#) 参数指定 event callback 函数。
[需要注意的是该 callback 是在协议栈的运行任务中执行的,所以不能在 callback 中执行复杂耗时的逻辑, 以免影响协议栈的运行效率。](#)

event callback 定义如下：

```
typedef int32_t(ieee80211_evt_cb)(uint8 ifidx, uint16 evt, uint32 param1, uint32 param2);
```

- 参数 ifidx: 产生事件的接口索引
- 参数 evt: 事件 id, 见枚举: enum ieee80211_event
- 参数 param1: event 参数 1, 具体含义需要查阅 event 列表说明
- 参数 param2: event 参数 2, 具体含义需要查阅 event 列表说明
- 返回值: 不同的 event, 返回值具有不同的含义, 具体需要查阅 event 列表说明。

4.1. WiFi 协议栈 Event 列表

4.1.1. IEEE80211_EVENT_PAIR_START

启动配对事件。当协议栈启动一键配对时产生此事件。

- param1: 0, 无含义
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.2. IEEE80211_EVENT_PAIR_SUCCESS

配对成功事件。在 AP 和 STA 配对成功时产生此事件。配对成功之后, 在停止配对之前, 协议栈会重复周期性产生此事件, 直到停止配对。

- param1: uint8* 类型, 此次配对成功的另一端设备的 MAC 地址。
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.3. IEEE80211_EVENT_PAIR_DONE

停止配对事件。停止配对时产生此事件。

- param1: uint8* 类型, 此次配对成功的另一端设备 MAC 地址。如果未配对成功, 则存储的是全 0 数据。
- param2: 表示配对过程是否协商了角色
 - ◆ 0: 未协商角色
 - ◆ -1: 已协商角色, 并且当前设备为从设备 (STA 模式)
 - ◆ 1: 已协商角色, 并且当前设备为主设备 (AP 模式)

- 返回值：无含义，返回 0 即可

4.1.4. IEEE80211_EVENT_SCAN_START

启动扫描事件。协议栈进入扫描状态时产生此事件。协议栈进入扫描状态可能是因为调用 `ieee80211_scan` API，也可能是因为协议栈自动扫描。

- `param1`：0，无含义
- `param2`：0，无含义
- 返回值：无含义，返回 0 即可

4.1.5. IEEE80211_EVENT_SCAN_DONE

扫描完成事件。协议栈扫描完成时产生此事件。产生此事件可能是因为停止扫描，或者扫描结束。

- `param1`：0，无含义
- `param2`：0，无含义
- 返回值：无含义，返回 0 即可

4.1.6. IEEE80211_EVENT_CONNECT_START

连接启动事件。协议栈开始连接 AP 时产生此事件，进入此事件表明协议栈已经发现 AP，通常是 STA 接口产生此事件。

- `param1`：0，无含义
- `param2`：0，无含义
- 返回值：无含义，返回 0 即可

4.1.7. IEEE80211_EVENT_CONNECTED

连接成功事件。协议栈连接 AP 成功后产生此事件。AP 接口产生此事件表明有 STA 成功连接自己，STA 接口产生此事件表明自己成功连接到 AP。

- param1: uint8* 类型。AP 接口时表示是新连接的 sta mac 地址，STA 接口时表示是连接的 AP mac 地址。
- param2: 0, 无含义
- 返回值: 无含义，返回 0 即可

4.1.8. IEEE80211_EVENT_CONNECT_FAIL

连接失败事件。通常是 STA 接口在多次尝试连接 AP 失败时产生此事件。

- param1: uint8* 类型。尝试连接的 AP mac 地址。
- param2: uint16 类型，连接失败时的 status code
- 返回值: 无含义，返回 0 即可

4.1.9. IEEE80211_EVENT_DISCONNECTED

断开连接事件。AP 接口产生此事件表示某个 sta 断开连接，STA 接口产生此事件表示自己与 AP 断开连接。

- param1: uint8* 类型。存储的断开连接的 AP mac 地址或 STA mac 地址。
- param2: uint16 类型，断开连接时的 [reason code](#)。
- 返回值: 无含义，返回 0 即可

4.1.10. IEEE80211_EVENT_RX_FRAME

接收数据的事件。协议栈在成功接收数据时产生此事件。接收到每一个数据都会产生此事件。

- param1: uint8* 类型，表示接收到的数据地址。注意：数据是 **WiFi 帧格式**。
- param2: 数据长度。
- 返回值：
 - ◆ 0: 通知协议栈继续接收该数据
 - ◆ 非 0: 通知协议栈 **丢弃** 该数据

4.1.11. IEEE80211_EVENT_RSSI

RSSI 信号变化事件。协议栈在检测到 RSSI 发生变化时产生此事件。协议栈会连续统计 10 个数据的接收信号，进行加权平均。如果信号波动超过阈值（默认是 6db）时就产生此事件。AP 接口产生此事件表示某个 sta 的接收信号发生波动，STA 接口产生此事件表示接收 AP 的信号发生波动。

- param1: int8 类型，表示本次接收信号的 rssi 值。
- param2: uint8* 类型，表示发生信号波动的 AP 或 STA 的 mac 地址。
- 返回值：无含义，返回 0 即可

使用 ieee80211_conf_set_signal_threshold API 可以修改信号波动阈值。

4.1.12. IEEE80211_EVENT_NEW_BSS

发现新 AP 的事件。协议栈找到 1 个新 AP 时产生此事件。AP 和 STA 接口都可以产生此事件，都可以执行扫描动作。

- param1: uint8* 类型, 表示 AP 的 bssid
- param2: uint8* 类型, 表示 AP 的 ssid
- 返回值: 无含义, 返回 0 即可

4.1.13. IEEE80211_EVENT_UPDATE_BSS

更新 AP 信息的事件。协议栈更新 AP 信息时产生此事件。AP 和 STA 接口都可以产生此事件, 都可以执行扫描动作。

- param1: uint8* 类型, 表示 AP 的 bssid
- param2: uint8* 类型, 表示 AP 的 ssid
- 返回值: 无含义, 返回 0 即可

4.1.14. IEEE80211_EVENT_STA_PS_START

sta 进入休眠的事件。AP 接口产生此事件, 当有 sta 进入休眠时协议栈就会产生此事件通知应用程序。

- param1: uint8* 类型, 表示进入休眠的 sta mac 地址
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.15. IEEE80211_EVENT_STA_PS_END

sta 退出休眠的事件。AP 接口产生此事件, 当有 sta 退出休眠时协议栈就会产生此事件。

- param1: uint8* 类型, 表示退出休眠的 sta mac 地址

- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.16. IEEE80211_EVENT_INTERFACE_ENABLE

协议栈接口打开事件。执行 `ieee80211_iface_start` API 时会产生此事件。

- param1: 0, 无含义
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.17. IEEE80211_EVENT_INTERFACE_DISABLE

协议栈接口关闭事件。执行 `ieee80211_iface_stop` API 时会产生此事件。

- param1: 0, 无含义
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.18. IEEE80211_EVENT_ADD_CUSTOMER_IE

通知添加自定义 `element` 的事件。协议栈在发送管理帧之前会产生此事件, 应用程序可以在此事件向管理帧中添加自定义的 `element` 数据。

- param1: `uint8*` 类型, 本次发送的管理帧数据地址。可以通过此参数识别管理帧类型, 判断是否需要添加自定义 `element` 数据。
- param2: `uint8**` 类型, 输出参数。通知协议栈需要添加的 `element` 数据的地址。协议栈会将该参数指定的数据添加到管理帧中发送出去。

- 返回值: uint32 类型
- ◆ 0: 没有 element 数据,
- ◆ 非 0: 需要添加的 element 数据长度。

4.1.19. IEEE80211_EVENT_MAC_CHANGE

接口 MAC 地址更改事件。接口 MAC 地址被修改时产生此事件。

- param1: uint8* 类型, 修改后的 mac 地址
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可。

4.1.20. IEEE80211_EVENT_EVM

接收信号的 EVM 发送波动事件。协议栈在检测到 EVM 发生变化时产生此事件。协议栈会连续统计 10 个数据的接收信号 EVM, 进行加权平均。如果信号 EVM 波动超过 10 时就产生此事件。AP 接口产生此事件表示某个 sta 的接收信号 EVM 发生波动, STA 接口产生此事件表示接收 AP 的信号 EVM 发生波动。

- param1: int8 类型, 表示本次接收信号的 evm 值。
- param2: uint8* 类型, 表示发生 EVM 波动的 AP 或 STA 的 mac 地址。
- 返回值: 无含义, 返回 0 即可

4.1.21. IEEE80211_EVENT_PRE_AUTH

STA 请求 auth 认证事件。AP 接口接收到 sta auth 请求时产生此事件。

- param1: uint8* 类型, 发起 auth 请求的 sta mac 地址

- param2: 0, 无含义
- 返回值: 通知协议栈是否接受该 sta 的 auth 请求。
 - ◆ 0: 允许 sta 继续连接,
 - ◆ 非 0: 拒绝该 sta 连接, 并表示拒绝的原因。

4.1.22. IEEE80211_EVENT_PRE_ASSOC

STA 请求连接事件。AP 接口接收到 sta assoc 请求时产生此事件。

- param1: uint8* 类型, 发起 assoc 请求的 sta mac 地址
- param2: 0, 无含义
- 返回值: 通知协议栈是否接受该 sta 的 assoc 请求。
 - ◆ 0: 允许 sta 继续连接,
 - ◆ 非 0: 拒绝该 sta 连接, 并表示拒绝的原因。

4.1.23. IEEE80211_EVENT_UNPAIR_SUCCESS

配对解绑事件。协议栈收到解绑请求时产生此事件。AP 接口和 STA 接口都可以发起解绑请求, 所以 AP 和 STA 接口都会产生此事件。

- param1: uint8* 类型, 发起解绑请求的 sta mac 地址
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可。

4.1.24. IEEE80211_EVENT_TX_BITRATE

协议栈预估的传输速率。协议周期性产生此事件, 通知应用程序当前链路预估的

可最大传输速率，单位的 Kbps。应用程序可以根据此事件调整自身的行为，例如对于视频传输，可以动态调整视频码率。

- param1: uint32 类型，预估的最大传输速率，单位 kbps。
- param2: 0，无含义
- 返回值：无含义，返回 0 即可。

4.1.25. IEEE80211_EVENT_TX_FRAME

协议栈 TX 数据之前产生此事件，通知应用程序将要 TX 一笔数据。每次 TX 数据都会产生此事件，应用程序可以在此事件中根据数据内容执行一些特定的动作。

- param1: uint8* 类型，表示将要 TX 的数据地址。数据是 [WiFi 帧格式](#)。
- param2: 无效
- 返回值：
 - ◆ 0：通知协议栈继续 TX 该数据
 - ◆ 非 0：通知协议栈 [丢弃](#) 该数据

4.1.26. IEEE80211_EVENT_RX_CUSTMGMT

接收到自定义管理帧事件。当协议栈接收到泰芯自定义管理帧时产生此事件。发送方使用 ieee80211_tx_custmgmt API 发送自定义管理帧，接收方就会产生此事件。

- param1: struct ieee80211_custmgmt_data * 类型。

```
struct ieee80211_custmgmt_data{
    uint8 *from; //发送方的 mac 地址
    uint8 *data; //接收到的数据
    uint8 len; //数据长度
};
```
- param2: 0，无含义
- 返回值：无含义，返回 0 即可

4.1.27. IEEE80211_EVENT_WRONG_KEY

密码错误提示事件。当协议栈尝试连接 AP 失败时，会判断是否因为密码错误，产生此事件通知应用程序。

- param1: 0, 无含义
- param2: 0, 无含义
- 返回值: 无含义, 返回 0 即可

4.1.28. IEEE80211_EVENT_CHANNEL_CHANGE

信道切换事件。当协议栈检测到信道发生切换时产生此事件。产生信道切换的原因可能是 AP 通知切换信道，或因检测到当前信道有干扰而切换信道。AP 和 STA 都会产生此事件。

- param1: uint8 类型, 切换信道之前的[旧信道](#)
- param2: uint8 类型, 切换信道之后的[新信道](#)
- 返回值: 无含义, 返回 0 即可

4.1.29. IEEE80211_EVENT_TX_STATUS

数据 TX 状态事件。当协议栈发送数据完成后产生此事件，通知应用程序该数据发送成功或失败。发送的每一笔数据都产生此事件。

- param1: uint8* 类型, 表示本次发送的数据地址, 数据格式为 WiFi 帧格式。
- param2: uint8 类型, 表示是否成功, 0: 发送失败, 1: 发送成功。
- 返回值: 无含义, 返回 0 即可

4.1.30. IEEE80211_EVENT_PAIR_NGO

配对协商角色事件。在配对时如果 2 个设备的角色相同，则可以进行协商，决定谁做 AP，谁做 STA。默认的协商规则是：比较 2 个设备的 MAC 地址，转成 64bit 数字，大的做 AP，小的做 STA。

在此过程会产生 IEEE80211_EVENT_PAIR_NGO 事件，通知应用程序正在进行配对协商。应用程序可以通过此事件自定义协商规则，通过该事件的返回值控制协商结果。

- param1: uint8* 类型，表示请求配对的设备 MAC 地址。
- param2: 0，无含义。
- 返回值：
 - ◆ 1: 本设备做 AP
 - ◆ -1: 本设备做 STA
 - ◆ 其他值: 应用程序未处理此事件

注：通过 index 参数可以知道对方的角色，因为只有 2 个设备的角色相同时才会产生此事件。

4.2. WiFi 协议栈 sysevt 机制

如 4.1 章节所说, 由于在协议栈的 event callback 函数中不能执行复杂耗时的逻辑, 所以有些 event 需要配合 sysevt 模块来执行一些复杂耗时的逻辑, 例如读写 flash。sysevt 机制的介绍在 SDK 入门指南第 9 章节。

sysevt.h 定义了一些和 WiFi 有关的 event, 每个 event 都可以携带一个 event data, event data 的具体含义是随 event id 变化。

- **SYSEVT_WIFI_CONNECT_START**

STA 接口开始连接, event data: 0

- **SYSEVT_WIFI_CONNECTED**

STA 接口连接成功, event data: 自己的 AID.

- **SYSEVT_WIFI_CONNECT_FAIL**

STA 接口连接失败, event data: 连接失败的状态码。

- **SYSEVT_WIFI_DISCONNECT**

STA 接口断开连接, event data: 断开连接的原因码

- **SYSEVT_WIFI_SCAN_START**

开始扫描, event data: 0

- **SYSEVT_WIFI_SCAN_DONE**

扫描完成, event data: 0

- **SYSEVT_WIFI_STA_DISCONNECT**

AP 接口检测到 sta 断开连接, event data: 断开连接的 sta AID.

- **SYSEVT_WIFI_STA_CONNECTED**

AP 接口检测到 sta 连接成功, event data: 连接成功的 sta AID.

- **SYSEVT_WIFI_STA_PS_START**

AP 接口检测到 sta 进入休眠, event data: 进入休眠的 sta AID.

- **SYSEVT_WIFI_STA_PS_END**

AP 接口检测到 sta 退出休眠, event data: 退出休眠的 sta AID.

- **SYSEVT_WIFI_PAIR_DONE**

一键配对完成, event data: 配对是否成功, 1:success, 0:fail.

- **SYSEVT_WIFI_TX_SUCCESS**

WiFi 发送数据成功, event data: 数据标签, 该标签是通过 [ieee80211_conf_set_datatag](#) API 设置的。

- **SYSEVT_WIFI_TX_FAIL**

WiFi 发送数据失败, event data: 数据标签, 该标签是通过 [ieee80211_conf_set_datatag](#) API 设置的。

- **SYSEVT_WIFI_WRONG_KEY**

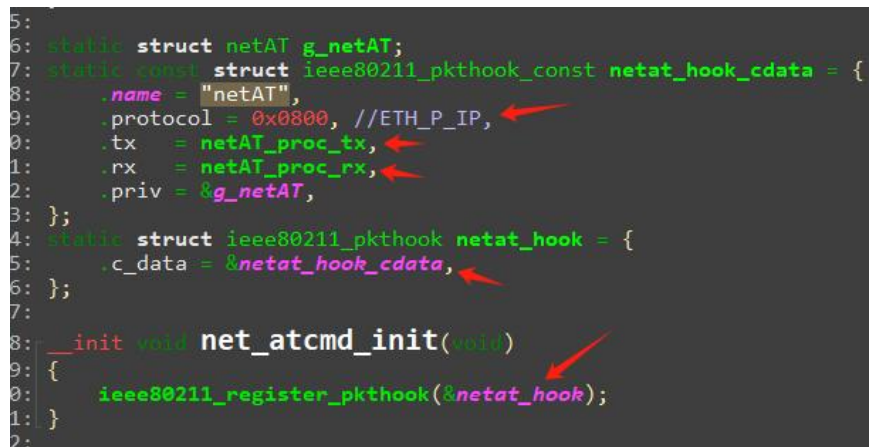
WiFi 连接发现密码错误, event data:0

5. WiFi 协议栈 Hook 机制

SDK WiFi 协议栈设计了 hook 机制，支持应用程序参与控制数据的发送和接收。

协议栈的 IEEE80211_EVENT_TX_FRAME 和 IEEE80211_EVENT_RX_FRAME 这 2 个事件也可以用于处理发送和接收的数据，但是 hook 机制的不同之处是 hook 处理的数据是以太网帧格式，对应用程序开发更加友好。

应用程序使用 [ieee80211_register_pkthook](#) API 向协议栈注册一个 hook，如下图所示：



```
5: static struct netAT g_netAT;
6: static const struct ieee80211_pkthook_const netat_hook_cdata = {
7:     .name = "netAT",
8:     .protocol = 0x0800, //ETH_P_IP,
9:     .tx = netAT_proc_tx,
10:    .rx = netAT_proc_rx,
11:    .priv = &g_netAT,
12: };
13: static struct ieee80211_pkthook netat_hook = {
14:     .c_data = &netat_hook_cdata,
15: };
16:
17:
18: __init void net_atcmd_init(void)
19: {
20:     ieee80211_register_pkthook(&netat_hook);
21: }
```

上图的示例代码注册了一个 hook: netAT，指定了 hook 需要处理的协议类型为 0x800（IP 包），设置了 hook 的 tx/rx 函数。

当 WiFi 协议栈发送或接收 IP 包时就会执行该 hook 的 tx/rx 函数，从而让该 hook 能够参与 IP 包的发送和接收过程。

通过 hook 机制，应用代码在数据收发过程中可以采取一些动作，例如：

- 修改 IP 包的数据内容
- 对 IP 包进行过滤：是丢弃还是继续发送
- 识别 IP 包数据内容，执行一些特定的应用代码，或给数据打个标签

应用程序注册 hook 时需要填充的数据类型是 struct ieee80211_pkthook_const，定义如下：

```
struct ieee80211_pkthook_const{
    const char *name; //hook 的名称
    void *priv;       //hook 的私有数据，在执行 tx/rx 函数时会传递回给 hook
    uint16 protocol;  //hook 关注的以太网协议类型
    uint16 mcast: 1,  //hook 是否关注所有的组播报
            ucast:1,  //hook 是否关注所有的单播包
            rev: 14;
    ieee80211_pkthdl tx; //TX 处理函数，协议栈 TX 数据时执行此函数
    ieee80211_pkthdl rx; //RX 处理函数，协议栈 RX 数据时执行此函数
};

typedef enum { //hook 的 tx/rx 函数返回值定义
    IEEE80211_PKTHDL_CONTINUE = 0, //TX/RX 继续执行
    IEEE80211_PKTHDL_CONSUMED = 1, //该数据已被消耗，通知协议栈丢弃数据
} ieee80211_pkthdl_res;

struct ieee80211_hookdata{ //hook data
    uint8 *data; //数据地址
    uint16 len;  //数据长度
    uint16 ext: 1, recv: 15;
    uint32 hdl;  //数据句柄，ieee80211\_conf\_set\_datatag API 需要这个句柄
};
```

6. WiFi 协议栈 API

SDK WiFi 协议栈的 API 定义在 sdk/include/umac/ieee80211.h 文件中。

6.1. 初始化 API

6.1.1. ieee80211_init

ieee80211_init API 用于初始化 WiFi 协议栈。

- **参数 param [in]** : WiFi 协议栈初始化参数

```
struct ieee80211_initparam {
    uint8    vif_maxcnt; //支持创建接口的最大个数
    uint8    bss_maxcnt; //支持缓存的 AP 信息最大个数
    uint8    headroom, tailroom;
    uint16   sta_maxcnt; //支持连接的 sta 的最大个数
    uint16   bss_lifetime; //缓存的 AP 信息生存时间
    uint16   stack_size; //协议栈的 task 堆栈大小, 默认是 2K
    uint16   no_rxtask:1, //协议栈是否使用独立的 rx task
            ssid_fuzzy_match:4, //协议栈是否开启 ssid 模糊匹配
            rev: 11;
    ieee80211_evt_cb evt_cb; //协议栈的 event callback 函数
};
```

- ap 信息生存时间: 当 AP 列表的数量超过 bss_maxcnt/2 时, ap 信息的生存时间自动调整为 5s, 直到清除后的 ap 数量小于 bss_maxcnt/2, 生存时间恢复为 bss_lifetime 值。

- **返回值** : 表示是否成功, 0: 成功, 非 0: 失败, errno

6.1.2. ieee80211_support_twx830x

ieee80211_support_twx830x API 用于初始化协议栈支持 TWX8301 芯片。

- **参数 ops [in]** : LMAC 协议栈的句柄
- **返回值** : 表示是否成功, 0: 成功, 非 0: 失败

6.1.3. ieee80211_support_txw80x

ieee80211_support_txw80x API 用于初始化协议栈支持 TXW80x 芯片。

- 参数 ops [\[in\]](#) : LMAC 协议栈的句柄
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.1.4. ieee80211_support_txw81x

ieee80211_support_txw81x API 用于初始化协议栈支持 TXW81x 芯片。

- 参数 ops [\[in\]](#) : LMAC 协议栈的句柄
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.1.5. ieee80211_iface_create_ap

ieee80211_iface_create_ap API 用于创建标准协议 AP 模式接口。

- 参数 ifidx [\[in\]](#) : AP 模式接口使用的接口索引
- 参数 band [\[in\]](#) : WiFi 频段类型
 - IEEE80211_BAND_2GHZ: 2.4G 频段, 适用 TXW80x, TXW81x 芯片
 - IEEE80211_BAND_5GHZ: 5G 频段
 - IEEE80211_BAND_60GHZ: 60G 频段
 - IEEE80211_BAND_S1GHZ: 低于 1G 频段, 适用 TXW8301 芯片
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.1.6. ieee80211_iface_create_sta

ieee80211_iface_create_sta API 用于创建标准协议 STA 模式接口。

- 参数 ifidx [\[in\]](#) : STA 模式接口使用的接口索引
- 参数 band [\[in\]](#) : WiFi 频段类型

- IEEE80211_BAND_2GHZ: 2.4G 频段, 适用 TXW80x, TXW81x 芯片
- IEEE80211_BAND_5GHZ: 5G 频段
- IEEE80211_BAND_60GHZ: 60G 频段
- IEEE80211_BAND_S1GHZ: 低于 1G 频段, 适用 TXW8301 芯片
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.1.7. ieee80211_iface_create_wnbap

ieee80211_iface_create_wnbap API 用于创建泰芯私有协议 WNBAP 模式接口。

- 参数 ifidx [[in](#)] : WNBAP 模式接口使用的接口索引
- 参数 band [[in](#)] : WiFi 频段类型
 - IEEE80211_BAND_2GHZ: 2.4G 频段, 适用 TXW80x, TXW81x 芯片
 - IEEE80211_BAND_5GHZ: 5G 频段
 - IEEE80211_BAND_60GHZ: 60G 频段
 - IEEE80211_BAND_S1GHZ: 低于 1G 频段, 适用 TXW8301 芯片
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.1.8. ieee80211_iface_create_wnbsta

ieee80211_iface_create_wnbsta 用于创建泰芯私有协议 WNBSTA 模式接口。

- 参数 ifidx [[in](#)] : WNBSTA 模式接口使用的接口索引
- 参数 band [[in](#)] : WiFi 频段类型
 - IEEE80211_BAND_2GHZ: 2.4G 频段, 适用 TXW80x, TXW81x 芯片
 - IEEE80211_BAND_5GHZ: 5G 频段
 - IEEE80211_BAND_60GHZ: 60G 频段
 - IEEE80211_BAND_S1GHZ: 低于 1G 频段, 适用 TXW8301 芯片

- 返回值：表示是否成功，0：成功，非 0：失败

6.1.9. ieee80211_iface_create_p2pdev

ieee80211_iface_create_p2pdev API 用于创建标准协议 p2p 模式接口。

- 参数 ifidx [in]：p2p 模式接口使用的接口索引
- 参数 band [in]：WiFi 频段类型
 - IEEE80211_BAND_2GHZ：2.4G 频段，适用 TXW80x, TXW81x 芯片
 - IEEE80211_BAND_5GHZ：5G 频段
 - IEEE80211_BAND_60GHZ：60G 频段
 - IEEE80211_BAND_S1GHZ：低于 1G 频段，适用 TXW8301 芯片
- 返回值：表示是否成功，0：成功，非 0：失败

6.1.10. ieee80211_iface_start

ieee80211_iface_start API 用于启动已创建的接口。

- 参数 ifidx [in]：需要启动的接口索引
- 返回值：表示是否成功，0：成功，非 0：失败

6.1.11. ieee80211_iface_stop

ieee80211_iface_stop API 用于停止已启动的接口。

- 参数 ifidx [in]：需要停止的接口索引
- 返回值：表示是否成功，0：成功，非 0：失败

6.1.12. ieee80211_deliver_init

ieee80211_deliver_init API 用于初始化协议栈的转发路由表。

- 参数 max [in] : 路由表缓存信息的最大数量
- 参数 lifetime [in]: 路由表缓存信息的生成时间, 单位: 秒
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.1.13. ieee80211_conf_stabr_table

ieee80211_conf_stabr_table API 用于初始化 WiFi 协议栈的 STA 接口桥接路由表,

该 API 仅对 STA 接口有效。

- 参数 ifidx [in]: 接口索引
- 参数 max [out]: 路由表信息最大数量
- 参数 lifetime [in]: 路由表信息的生存时间
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.1.14. wpa_passphrase

wpa_passphrase API 用于根据 ssid 和明文密码 password, 计算密文密码 psk。

- 参数 ssid [in] : WiFi ssid
- 参数 passwd [in]: WiFi password, 明文密码
- 参数 psk [out] : 生成的密文密码 psk
- 返回值 : 表示是否成功, 0: 成功, -1: 失败

6.2. 基础配置 API

6.2.1. ieee80211_conf_set_channel

ieee80211_conf_set_channel API 用于设置 WiFi 信道。

该 API 对 sta 模式无效，因为 sta 需要扫描连接 AP，跟随 AP 的 channel。

- 参数 ifidx [\[in\]](#)：接口索引
- 参数 channel [\[in\]](#)：设置新的 channel
- 返回值：表示是否成功，0：成功，非 0：失败。

6.2.2. ieee80211_conf_set_ssid

ieee80211_conf_set_ssid API 用于设置 WiFi ssid 参数。

- 参数 ifidx [\[in\]](#)：接口索引
- 参数 ssid [\[in\]](#)：WiFi SSID 参数。
- 返回值：表示是否成功，0：成功，非 0：失败。

6.2.3. ieee80211_conf_set_keymgmt

ieee80211_conf_set_keymgmt API 用于设置 WiFi 加密模式。

- 参数 ifidx [\[in\]](#)：接口索引
- 参数 keymgmt [\[in\]](#)：WiFi 加密模式，有效值如下：
#define WPA_KEY_MGMT_PSK BIT(1)
#define WPA_KEY_MGMT_NONE BIT(2)
#define WPA_KEY_MGMT_SAE BIT(10)
#define WPA_KEY_MGMT_OWE BIT(22)
- 返回值：表示是否成功，0：成功，非 0：失败。

6.2.4. ieee80211_conf_set_psk

ieee80211_conf_set_psk API 用于设置 WiFi 加密使用的密文密码。

密文密码通常是使用 wpa_passphrase API 计算得到的，但是也可以完全自定义，例如使用随机数。

- 参数 ifidx [in]: 接口索引
- 参数 psk [in]: 密文密码
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.2.5. ieee80211_conf_set_passwd

ieee80211_conf_set_passwd API 用于设置 WiFi 加密的明文密码，在开启 wpa3 时需要使用此 API。

- 参数 ifidx [in]: 接口索引
- 参数 passwd [in]: 明文密码
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.2.6. ieee80211_conf_set_beacon_int

ieee80211_conf_set_beacon_int API 用于修改 AP 接口的 beacon 周期。默认 beacon 周期为 100ms。

- 参数 ifidx [in]: 接口索引
- 参数 beacon_int [in]: 新的 beacon 周期, 单位: 毫秒
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3. 高级配置 API

6.3.1. ieee80211_conf_set_bssbw

ieee80211_conf_set_bssbw API 用于设置 WiFi 协议栈的 BSS 带宽。

- 参数 ifidx [in]: 接口索引
- 参数 bssbw [in]: BSS 带宽, 设置前需要确认芯片是否支持此带宽。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.2. ieee80211_conf_set_dtim_int

ieee80211_conf_set_dtim_int API 用于设置 AP 接口的 DTIM 周期。默认 DTIM 周期是 10, 也就是 dtim10, 表示 STA 在休眠时每 10 个 beacon 时刻醒来一次。

- 参数 ifidx [in]: 接口索引
- 参数 dtim_int [in]: 新的 DTMI 周期, 该值的含义是 beacon 周期的个数
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.3. ieee80211_conf_set_bss_max_idle

ieee80211_conf_set_bss_max_idle API 用于设置 AP 接口的 max idle 参数。该参数要求 sta 在 max idle 时间内至少通信一次, 让 AP 判断 sta 是否在线。

- 参数 ifidx [in]: 接口索引
- 参数 max_idle_period [in]: max idle 时间, 单位: 秒, 默认是 300 秒。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.4. ieee80211_conf_set_bssid

ieee80211_conf_set_bssid API 用于设置 WiFi 连接的 AP BSSID。当存在多个 SSID 相同的 AP 时，如果需要指定连接某个 AP，需要使用此 API 进行设置，否则协议栈将根据 SSID 查找 AP，随机选择其中 1 个 AP 进行连接。

- 参数 ifidx [in]: STA 接口索引
- 参数 bssid [in]: AP 的 mac 地址
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

注: 使用此功能, 需要先设置 ssid, 再设置 bssid。

6.3.5. ieee80211_conf_set_mac

ieee80211_conf_set_mac API 用于修改接口的 MAC 地址。

- 参数 ifidx [in]: 接口索引
- 参数 mac [in]: 设置新的 MAC 地址
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.6. ieee80211_conf_set_aphide

ieee80211_conf_set_aphide API 用于设置 AP 接口隐藏 SSID。隐藏 SSID 的 AP 不会被设备扫描到, 除非设备在扫描时指定了 ssid 参数。

- 参数 ifidx [in]: 接口索引
- 参数 hide [in]: 0: 关闭隐藏, 1: 开启隐藏。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.7. ieee80211_conf_set_hwmode

ieee80211_conf_set_hwmode 用于设置 WiFi 协议模式。

```
enum ieee80211_hwmode {
    IEEE80211_HWMODE_NONE,
    IEEE80211_HWMODE_11B,    // b only
    IEEE80211_HWMODE_11G,    // bg mix
    IEEE80211_HWMODE_11N,    // bgn mix
    IEEE80211_HWMODE_11AH,   // ah only
};
```

- 参数 ifidx [in]: 接口索引
- 参数 mode [in]: WiFi 协议模式: enum ieee80211_hwmode。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.8. ieee80211_expand_mlme_size

ieee80211_expand_mlme_size API 用于扩展 WiFi 协议栈的管理帧长度。当需要在管理帧中添加自定义 element 时, 可能会遇到管理帧 buffer 长度不够的情况, 可以通过此 API 通知协议栈扩展管理帧长度, 便于添加自定义的 element 信息。

- 参数 ifidx [in]: 接口索引
- 参数 size [in]: 需要扩展 size 的管理帧信息。

```
/*MLME frame expand size*/
struct ieee80211_mlme_size{
    uint16 probe req, probe resp;
    uint16 beacon, vendor spec action;
    uint16 assoc req, assoc resp;
    uint8  auth, deauth, disassoc, addba_resp;
    uint16 p2p_neg;
};
```

每个字段对应一种管理帧的扩展 size, 表示需要协议栈在创建此管理帧时扩大指定的 size。

- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.9. ieee80211_conf_set_wpa_cipher

ieee80211_conf_set_wpa_cipher API 用于修改协议栈的 WiFi 加密参数。

- 参数 ifidx [in]: 接口索引。
- 参数 wpa_proto [in]: WPA 协议版本
- 参数 pairwise_cipher [in]: 获取的 AP 的加密信息。
- 参数 group_cipher [in]: 获取的 AP 的加密信息。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

■ 参数 wpa_proto 取值如下:

```
#define WPA_PROTO_WPA      BIT(0)
#define WPA_PROTO_RSN      BIT(1)
```

● 参数 pairwise_cipher 和 group_cipher 取值如下:

```
#define WPA_CIPHER_NONE    BIT(0)
#define WPA_CIPHER_TKIP    BIT(3)
#define WPA_CIPHER_CCMP    BIT(4)
#define WPA_CIPHER_CCMP_256 BIT(9)
```

6.3.10. ieee80211_conf_set_isolate

ieee80211_conf_set_isolate API 用于设置接口隔离。WiFi 协议栈默认允许所有接口之间相互转发通信。该 API 可以设置接口处于隔离状态, 不能和其他接口转发通信。

- 参数 ifidx [in]: 接口索引
- 参数 isolate [in]: 0: 关闭隔离, 1: 开启隔离
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.11. ieee80211_conf_set_chanlist

ieee80211_conf_set_chanlist API 用于设置自定义信道列表。执行此 API 将会修改 WiFi 的信道列表, 可以完全自定义信道列表。

- 参数 ifidx [in]: 接口索引
- 参数 chan_list [in]: 信道列表, 要求是 uint16 数组。
- 参数 count [in]: 信道数量
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.12. ieee80211_conf_set_wpa_group_rekey

ieee80211_conf_set_wpa_group_rekey API 用于设置 AP 接口的组播 key 更新周期。

- 参数 ifidx [in]: 接口索引
- 参数 wpa_group_rekey [in]: 更新周期, 单位: 秒。设置 0 表示关闭组播 key 更新功能。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.13. ieee80211_conf_set_dupfilter

ieee80211_conf_set_dupfilter API 用于设置 WiFi 协议栈是否开启重复包过滤功能。

- 参数 ifidx [in]: 接口索引
- 参数 enable [in]: 0: 关闭重复包过滤, 1: 开启重复包过滤
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.14. ieee80211_conf_set_conn_timeout

ieee80211_conf_set_conn_timeout API 用于修改 WiFi 协议栈的连接超时时间。

- 参数 ifidx [in]: 接口索引
- 参数 auth_tmo [in]: auth 阶段的 timeout 值, 默认值是 100ms。

- 参数 `assoc_tmo` [in]: `assoc` 阶段的 `timeout` 值, 默认值是 100ms。
- 参数 `handshake_tmo` [in]: 4 次握手阶段的 `timeout` 值, 默认是 2000ms。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.15. `ieee80211_conf_set_aplost_time`

`ieee80211_conf_set_aplost_time` API 用于设置 STA 接口检测 AP 离线的时间。超过设置的时间未收到 AP 的任何数据, 则认为 AP 已离线。

- 参数 `ifidx` [in]: 接口索引
- 参数 `time` [in]: 检测 AP 离线的时间。默认值 30, 表示 30 个 beacon 周期。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.16. `ieee80211_conf_set_wmm_enable`

`ieee80211_conf_set_wmm_enable` API 用于设置 WiFi 协议栈是否开启 QoS 功能, 该功能默认是开启状态。

- 参数 `ifidx` [in]: 接口索引
- 参数 `enable` [in]: 0: 关闭 QoS, 1: 开启 QoS。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.17. ieee80211_conf_set_wmm_param

ieee80211_conf_set_wmm_param API 用于修改 WMM EDCA 参数。修改 EDCA 参数通常是 4 个 AC 一起修改，所以该 API 需要执行 4 次。

- 参数 ifidx [in]: 接口索引
- 参数 ac [in]: EDCA AC 队列, 取值范围: 0, 1, 2, 3, 对应: AC_VO, AC_VI, AC_BK, AC_BE
- 参数 param [in]: 指定某个 AC 的 EDCA 参数, 参数定义如下:

```
struct ieee80211_wmm_param {  
    uint16 txop:    //连续发送的时间上限, 0: 只发送一帧, 单位:us  
    uint16 cw_min:  //竞争窗口指数的最小值  
    uint16 cw_max:  //竞争窗口指数的最大值  
    uint8  aifsn:   //仲裁帧间隙数  
    uint8  acm;     //暂未使用  
};
```

对 AP 接口来说, 参数 ac 有 2 种情况:

- ac 参数取值: [0xf0, 0xf1, 0xf2, 0xf3], 表示只修改自己的本地 edca 参数, 不影响 sta 的 edca 参数。
- ac 参数取值: [0, 1, 2, 3], 表示修改 sta 的 edca 参数。执行此 API 会修改 AP 发送的 beacon 帧中的 wmm 参数, sta 会依此更新自己的 edca 参数。

对于 STA 接口来说, 执行此 API 只修改本地 EDCA 参数, 同时会拒绝更新 AP 发送的 EDCA 参数。

6.3.18. ieee80211_conf_set_use4addr

ieee80211_conf_set_use4addr API 用于设置接口是否开启 4 地址模式。AP 接口可以自动兼容 4 地址 sta 和非 4 地址 sta。

- 参数 ifidx [in]: 接口索引
- 参数 enable [in]: 0: 关闭 4 地址模式, 1: 开启 4 地址模式。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.19. ieee80211_conf_set_psdata_cnt

ieee80211_conf_set_psdata_cnt API 用于设置 AP 为休眠 sta 缓存数据的最大数量。
缓存数据超过最大数量时，丢弃旧的数据，缓存新的数据。

- 参数 ifidx [in]: 接口索引
- 参数 max_cnt [in]: 允许缓存数据的最大数量。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.20. ieee80211_conf_set_datatag

ieee80211_conf_set_datatag API 用于给特定的数据设置标签, 标签可以是 uint32 类型的任意值, 只要应用程序自己能识别即可。

该 API 通常是结合 hook 机制使用, 在 hook 中识别特定数据后, 给该数据打上标签。
WiFi 协议栈在发送该数据后会根据发送成功或失败, 产生 SYSEVT_WIFI_TX_SUCCESS 或 SYSEVT_WIFI_TX_FAILURE 事件。这 2 个事件的 event_data 就是该数据的标签值。

- 参数 ifidx [in]: 接口索引
- 参数 hdl [in]: 数据 handle, 这个值来自 struct ieee80211_hookdata。在执行 hook 的 tx/rx 函数时可以获取这个值。
- 参数 tag [in]: 数据标签。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.3.21. ieee80211_conf_set_signal_threshold

ieee80211_conf_set_signal_threshold API 用于修改 RSSI/EVM 波动阈值, 会影响 IEEE80211_EVENT_RSSI 和 IEEE80211_EVENT_EVM 事件的上报。

WiFi 协议栈会实时监测当前 AP 或 STA 的 RSSI/EVM 波动情况。当前后的 RSSI/EVM 波动差值大于设定的阈值后,就会产生 IEEE80211_EVENT_RSSI/IEEE80211_EVENT_EVM 事件。

RSSI 波动阈值默认是 6db, EVM 波动阈值默认是 6db, 通过此 API 可以修改这 2 个阈值。

- 参数 ifidx [in] : 接口索引
- 参数 rssi_thres [in] : rssi 波动阈值。
- 参数 evm_thres [in] : evm 波动阈值
- 返回值 : 表示设置是否成功, 0: 成功, 非 0: 失败

6.4. 一键配对 API

6.4.1. ieee80211_pair_enable

ieee80211_pair_enable API 用于初始化协议栈的一键配对功能。

- 参数 ifidx [in] : 启用配对功能的接口索引
- 参数 pair_magic [in] : 配对功能的 magic number, 在配对时相同 magic number 的设备才能相互配对成功。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.4.2. ieee80211_pairing

ieee80211_pair_enable API 用于控制一键配对功能的开/关。

- 参数 ifidx [in] : start/stop 配对功能的接口索引
- 参数 pair_magic [in] : 0: 停止配对, 1: 启动配对, 大于 1: 启动配对, 并修改 magic number。

- 返回值：表示是否成功，0：成功，非 0：失败

6.4.3. ieee80211_unpair

ieee80211_unpair API 用于执行配对信息解绑。

- 参数 ifidx [in]：配对解绑的接口索引
- 参数 mac [in]：需要解绑的设备 MAC 地址，AP 和 STA 都可以执行此 API 发起解绑操作，mac 参数是对方的 MAC 地址。
- 返回值：表示是否成功，0：成功，非 0：失败

6.4.4. ieee80211_conf_set_pair_ngo

ieee80211_conf_set_pair_ngo API 用于开启配对角色协商功能。

配对协商功能是指 2 个相同角色的设备配对时，协商谁做 AP，谁做 STA。

默认的协商规则是通过比较 MAC 地址大小：大的做 AP，小的做 STA。

触发此功能时会产生 [IEEE80211 EVENT PAIR NGO](#) 事件，通过此事件可以自定义协商规则。

此功能默认是开启的，可以通过该 API 关闭此功能。

- 参数 ifidx [in]：配对解绑的接口索引
- 参数 enable[in]：是否启用配对协商功能。
- 返回值：表示是否成功，0：成功，非 0：失败

6.5. 连接扫描 API

6.5.1. ieee80211_scan

ieee80211_scan API 用于控制协议栈进行扫描。

- 参数 ifidx [in] : 执行扫描动作的接口索引
- 参数 start [in] : 1: 启动扫描, 0: 停止扫描, 在扫描完成时也会自动停止。
- 参数 scan_param [in]: 扫描控制参数:

```
struct ieee80211_scandata {  
    uint32 chan_bitmap; //需要扫描的 channel bitmap, 每个 bit 位对应一个 channel, bit 位为 1, 表示需要扫描该 channel。channel_bitmap=0, 则扫描所有的 channel  
    uint8 ssid[SSID_MAX_LEN+1]; //指定扫描特定的 ssid  
    uint8 scan_time, //每个 channel 的扫描时间, 单位: 毫秒, 默认是 20ms  
        scan_cnt; //每个 channel 扫描时发送 probe request 的次数, 默认  
        发送 2 次。  
    uint8 passive_scan: 1, //是否执行被动扫描, 不发送 probe request  
        flush: 1, //启动扫描时是否清空 AP 列表  
        rev: 6;  
};
```

- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

注意: 该 API 仅启动协议栈开始扫描, 该 API 返回不代表协议栈已经完成扫描。扫描完成时会产生 SYSEVT_WIFI_SCAN_DONE 事件。接收 SYSEVT_WIFI_SCAN_DONE 事件后才能获取 AP 信息列表。

6.5.2. ieee80211_get_bsslist

ieee80211_get_bsslist API 用于从 WiFi 协议栈获取 AP 列表。

- 参数 bsslist [out]: 接收 AP 列表信息的 buffer 地址
- 参数 list_size [in]: buffer size
- 参数 ignore_dis [in]: 指示是否忽略被 disable 的 AP 信息。
- 返回值 : 获取到的 AP 信息数量。

注意: AP 信息列表存在老化时间。当 AP 列表个数大于 bss_maxcnt 的二分之一时老化时间自动调整为 5s, 否则老化时间为 bss_lifetime。老化时间到达后会清除部分超

时的 AP 信息。bss_maxcnt 和 bss_lifetime 是 WiFi 初始化时设置的参数。

6.5.3. ieee80211_cleanup_bsslist

ieee80211_cleanup_bsslist API 用于清空 WiFi 协议栈缓存的 AP 列表信息。

- 返回值：表示是否成功，0：成功，非 0：失败

6.5.4. ieee80211_conf_set_auto_assoc

ieee80211_conf_set_auto_assoc API 用于设置 WiFi 协议栈是否进行自动连接和自动扫描。协议栈默认是自动连接，自动扫描。使用此 API 可以关闭此行为，由应用程序控制协议栈的连接和扫描行为。

执行连接使用 ieee80211_start_connect API，执行扫描使用 ieee80211_scan API。

- 参数 ifidx [in]：接口索引。
- 参数 auto_assoc [in]：0：关闭自动连接，1：开启自动连接
- 参数 auto_scan [in]：0：关闭自动扫描，1：开启自动扫描
- 返回值：表示是否成功，0：成功，非 0：失败。

6.5.5. ieee80211_start_connect

ieee80211_start_connect API 用于控制 WiFi 协议栈发起连接。通常是在关闭自动连接的情况下使用此 API。关闭自动连接后，此 API 执行一次，WiFi 协议栈就执行一次连接。不管是否连接成功，WiFi 协议栈就会停下来，不会再次自动发起连接。

- 参数 ifidx [in]：接口索引。
- 返回值：表示是否成功，0：成功，非 0：失败。

6.5.6. ieee80211_disassoc

ieee80211_disassoc API 用于控制 WiFi 协议栈断开连接。执行此 API 可以断开连接，但是 sta 可能会又再次发起连接，只要 ssid/密码是正确的，sta 依然可以再次连接成功。

- 参数 ifidx [in]: 接口索引
- 参数 addr [in]: 需要断开的 sta mac 地址
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.5.7. ieee80211_disassoc_all

ieee80211_disassoc_all API 用于控制 WiFi 协议栈断开所有 sta 连接。

- 参数 ifidx [in]: 接口索引
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.5.8. ieee80211_bss_add_manualAP

ieee80211_bss_add_manualAP API 用于向协议栈注册 AP 信息, 通常是用于快速连接。应用代码将上次连接的 AP 信息保存到 flash, 开机时使用 flash 存储的 AP 信息向协议栈注册 AP 信息, 可以让协议栈跳过扫描的过程, 直接发起连接, 减少连接过程的时间。

- 参数 ssid [in]: 添加的 AP ssid
- 参数 bssid [in]: 添加的 AP 的 mac 地址
- 参数 channel [in]: AP 所在的信道
- 参数 band [in]: AP 的频段信息
- 参数 wpadata [in]: AP 的加密信息, 这个值保存时使用 ieee80211_bss_get_wpadata API 从协议栈中获取。

- 返回值：表示是否成功，0：成功，非 0：失败。

6.5.9. ieee80211_conf_set_scan_max

WiFi 协议栈在连接扫描 AP 过程中，如果连续扫描多次（所有信道扫描一遍为 1 次，默认是 15 次）都未发现 AP，就会上报 IEEE80211_EVENT_CONNECT_FAIL 事件，status code 是 WLAN_REASON_AP_NOT_FOUND。

ieee80211_conf_set_scan_max API 用于控制协议栈在扫描多少次后再上报 IEEE80211_EVENT_CONNECT_FAIL 事件。

- 参数 ifidx [in]：接口索引
- 参数 scan_max[in]：扫描次数
- 返回值：表示是否成功，0：成功，非 0：失败

6.6. 数据收发 API

6.6.1. ieee80211_scatter_tx

ieee80211_scatter_tx API 用于发送非连续的片段数据，主要是对接 LWIP 时使用。数据必须是以太网帧格式。

- 参数 ifidx [in]：发送数据的接口索引
- 参数 data [in]：scatter_data 数组地址
- 参数 count [in]：scatter_data 数组的大小
- 返回值：表示是否成功，0：成功，非 0：失败

6.6.2. ieee80211_tx

ieee80211_tx API 用于发送连续的数据，数据必须是以太网帧格式。

- 参数 ifidx [in] : 发送数据的接口索引
- 参数 data [in] : 数据地址。
- 参数 size [in] : 数据长度
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.6.3. ieee80211_input

ieee80211_input API 用于向 WiFi 协议栈输入一笔 RX 数据，借助 WiFi 协议栈的 RX 通道，向上输入到 lwip 或主控。

- 参数 ifidx [in] : 接口索引
- 参数 data [in] : 数据地址，数据必须是以太网帧格式。
- 参数 len [in] : 数据长度
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.6.4. ieee80211_tx_mgmt

ieee80211_tx_mgmt API 用于发送自定义管理帧。应用程序需要自行组装完整的 WiFi 管理帧再调用此 API 进行发送。

- 参数 ifidx [in] : 发送管理帧的接口索引
- 参数 data [in] : 管理帧数据地址，数据必须是完整的 WiFi 帧格式。MAC header 的 seq number 和 duration 填 0 即可。
- 参数 len [in] : 管理帧数据长度

- 返回值：表示是否成功，0：成功，非 0：失败

6.6.5. ieee80211_tx_custmgmt

ieee80211_tx_custmgmt API 用于发送自定义管理帧。为了简化自定义管理帧的发送，WiFi 协议栈提供了这个 API，使用此 API 可以不用关心 WiFi 帧格式如何组装。对 WiFi 帧格式不熟悉的情况下，也可以发送自定义管理帧。该 API 只要求提供任意格式的 payload 数据即可，协议栈会完成 WiFi 帧格式的封装。使用此 API 发送的管理帧，需要配合 IEEE80211_EVENT_RX_CUSTMGMT 来接收。

- 参数 ifidx [in]：发送管理帧的接口索引
- 参数 dest [in]：发送此管理帧的目的地址，可以是广播地址。
- 参数 data [in]：管理帧数据地址。可以是任意格式的数据，将作为管理帧 payload 发送。
- 参数 len [in]：管理帧数据长度
- 返回值：表示是否成功，0：成功，非 0：失败

6.7. Hook 和 Event API

6.7.1. ieee80211_register_pkthook

ieee80211_register_pkthook API 用于向 WiFi 协议栈注册 hook，hook 机制的介绍

请阅读[第 5 章节](#)。

- 参数 hook [in]: 需要注册的 hook 信息
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.7.2. ieee80211_event_cb

ieee80211_event_cb API 用于修改 WiFi 协议栈的 event callback 函数。执行此 API 将更新协议栈的 event callback 函数, [同时该函数将返回旧的 event callback 函数](#)。

应用程序在修改 event callback 时, 应当将旧的 callback 函数保存下来, 当执行完自己的 callback 函数应继续执行旧的 callback 函数, 避免丢失旧的 callback, 造成其他模块无法正常工作。

- 参数 cb [in]: 新的 event callback 函数
- 返回值 : 协议栈旧的 event callback 函数

6.8. 低功耗 API

6.8.1. ieee80211_conf_wakeup_sta

ieee80211_conf_wakeup_sta API 用于 AP 接口唤醒休眠的 STA, 并通知 STA 被唤醒的原因。

- 参数 ifidx [in]: 接口索引

- 参数 addr [in]: 被唤醒的 sta mac 地址
- 参数 reason [in]: 唤醒 sta 的原因
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.8.2. ieee80211_conf_get_wkreason

ieee80211_conf_get_wkreason API 用于获取协议栈接收到的唤醒原因。该唤醒原因通常是 AP 唤醒 sta 时, 由 AP 告知 sta 是什么原因唤醒它。该功能是泰芯自定义低功耗功能, 可以实现由 AP 在唤醒 STA 时通知 STA 唤醒原因。仅在 AP 和 STA 都是泰芯芯片的情况下使用。

- 参数 ifidx [in]: 接口索引
- 返回值 : 唤醒原因。0: 正常开机或 AP 未告知唤醒原因, 非 0: AP 告知的唤醒原因。

6.9. 信息获取 API

6.9.1. ieee80211_status

ieee80211_status API 用于打印协议栈的调试信息。

- 参数 buff [in] : 接收调试信息的换成 buffer
- 参数 size [in] : buffer size
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败

6.9.2. ieee80211_conf_get_psk

ieee80211_conf_get_psk API 用于获取 WiFi 协议栈使用的密文密码。

- 参数 ifidx [[in](#)]：接口索引
- 参数 psk [[out](#)]：接收 psk 的 buffer 地址，该 buffer 的 size 不能小于 32byte。
- 返回值：表示是否成功，0：成功，非 0：失败。

6.9.3. ieee80211_conf_get_keymgmt

ieee80211_conf_get_keymgmt API 用于获取 WiFi 协议栈当前使用的加密模式。

- 参数 ifidx [[in](#)]：接口索引
- 返回值：WiFi 加密模式。

6.9.4. ieee80211_conf_get_ssid

ieee80211_conf_get_ssid API 用于获取 WiFi 协议栈当前使用的 ssid 参数。

- 参数 ifidx [[in](#)]：接口索引
- 参数 ssid [[out](#)]：接收 SSID 参数的 buffer 地址，该 buffer 的 size 不能小于 32byte。

返回值：表示是否成功，0：成功，非 0：失败。

6.9.5. ieee80211_get_stalist

ieee80211_get_stalist API 用于从 WiFi 协议栈获取 sta 信息列表。

- 参数 ifidx [[in](#)]：接口索引

- 参数 `stalist` [out]: 接收 sta 列表信息的 buffer 地址
- 参数 `list_size` [in]: buffer size
- 返回值 : 获取到的 sta 信息数量。

6.9.6. `ieee80211_conf_get_bssbw`

`ieee80211_conf_get_bssbw` API 用于获取 WiFi 协议栈当前使用的 BSS 带宽。

- 参数 `ifidx` [in]: 接口索引
- 返回值 : WiFi 协议栈当前使用的 BSS 带宽。

6.9.7. `ieee80211_conf_get_bssid`

`ieee80211_conf_get_bssid` API 用于获取 WiFi 当前连接的 AP MAC 地址。AP 接口执行此 API 返回是 AP 接口的 MAC 地址。STA 接口执行此 API 返回的是 sta 连接的 ap mac 地址, 或者 sta 断开连接时返回是 `ieee80211_conf_set_bssid` API 设置的 bssid。

- 参数 `ifidx` [in]: STA 接口索引
- 参数 `bssid` [out]: bssid
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.9.8. `ieee80211_conf_get_channel`

`ieee80211_conf_get_channel` API 用于获取当前的 WiFi 信道。

- 参数 `ifidx` [in]: 接口索引
- 返回值 : 返回当前的 WiFi 信道。

6.9.9. ieee80211_conf_get_mac

ieee80211_conf_get_mac API 用于获取接口的 MAC 地址。

- 参数 ifidx [in]: 接口索引
- 参数 mac [out]: 获取的 MAC 地址
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.9.10. ieee80211_conf_get_connstate

ieee80211_conf_get_connstate API 用于获取当前的连接状态。

- 参数 ifidx [in]: 接口索引
- 返回值 : 当前连接状态, 返回值定义如下:

```
enum wpa_states {  
    WPA_DISCONNECTED,      //0 断开连接  
    WPA_INTERFACE_DISABLED, //1 接口已关闭  
    WPA_INACTIVE,          //2 关闭状态  
    WPA_SCANNING,          //3 扫描状态  
    WPA_AUTHENTICATING,    //4 发起认证状态  
    WPA_ASSOCIATING,       //5 发起连接状态  
    WPA_ASSOCIATED,        //6 已连接状态  
    WPA_4WAY_HANDSHAKE,    //7 4 次握手状态  
    WPA_GROUP_HANDSHAKE,   //8 组播 key 握手状态  
    WPA_COMPLETED,         //9 连接完成状态  
};
```

-

6.9.11. ieee80211_conf_get_stainfo

ieee80211_conf_get_stainfo API 用于从 WiFi 协议栈获取单个 sta 信息。可以根据 AID, 或 MAC 地址来获取 sta 信息。

- 参数 ifidx [in]: 接口索引

- 参数 aid [in]: 需要获取的 sta aid, aid 非 0 时根据 aid 查找 sta。
- 参数 mac [in]: 需要获取的 sta mac 地址, 当 mac 不是 NULL 值时, 优先则根据 mac 地址查找 sta。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.9.12. ieee80211_conf_get_stalist

ieee80211_conf_get_stalist API 用于从 WiFi 协议栈获取 sta 信息列表。

- 参数 ifidx [in]: 接口索引
- 参数 sta [out]: 接收 sta 信息列表的 buffer 地址
- 参数 count [in]: buffer 可存储 sta 信息的数量。
- 返回值 : 实际获取的 sta 数量。

6.9.13. ieee80211_conf_get_stacnt

ieee80211_conf_get_stacnt API 用于获取指定接口所连接的 sta 数量。

- 参数 ifidx [in]: 接口索引
- 返回值 : 该接口连接的 sta 数量。

6.9.14. ieee80211_conf_get_reason_code

ieee80211_conf_get_reason_code API 获取连接失败的 reason code。在连接失败时该 API 返回的 reason code 才是有效值。

- 参数 ifidx [in]: 接口索引
- 返回值 : reason code。

SDK 添加了部分自定义 reason code,在 sdk/include/lib/umac/ieee80211.h 文件中,说明如下:

```
/* txsemi custom reason code */
enum IEEE80211_TXREASON{
    WLAN_REASON_AP_NOT_FOUND = 60000, //未发现 AP
    WLAN_REASON_PAIR_START,           //启动配对断开连接
    WLAN_REASON_UNPAIR,               //解除配对绑定
    WLAN_REASON_RELAY_DISCONNECT,     //上级中继链路断开
};
```

6.9.15. ieee80211_conf_get_status_code

ieee80211_conf_get_status_code API 获取连接失败的 status code。在连接失败时该 API 返回的 status code 才是有效值。

- 参数 ifidx [in]: 接口索引
- 返回值 : status code。

6.9.16. ieee80211_chan_center_freq

ieee80211_chan_center_freq 用于获取指定 channel 的中心频点信息。

- 参数 ifidx [in]: 接口索引
- 参数 channel [in]: 信道号
- 返回值 : 指定信道的中心频点。

6.9.17. ieee80211_bss_get_wpdata

ieee80211_bss_get_wpdata API 用于获取当前连接的 AP 的加密信息,主要是用于将 AP 信息保存到 flash,加速开机连接过程。

- 参数 bssid [in]: 需要获取的 AP mac 地址, 根据 mac 地址查找 ap 信息。
- 参数 band [in]: AP 的频段信息
- 参数 wpadata [out]: 获取的 AP 的加密信息。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.10. WiFi 漫游 API

SDK WiFi 协议栈的标准协议和泰芯私有协议都可以支持快速漫游功能, 2 者之间的区别如下:

- 标准协议:
 - 优点: 可以兼容标准协议 802.11kvr 协议
 - 缺点: 协议栈的 STA 模式支持快速漫游, AP 模式不支持。STA 模式使用快速漫游时需要路由器支持 802.11kvr 协议才可以进行漫游。是否支持无缝漫游也取决于路由器。
- 私有协议:
 - 优点: AP 和 STA 模式均支持漫游, 且 AP 支持 STA 进行无缝漫游。
 - 缺点: 不能和标准协议兼容, 只能是私有协议的 AP 和 STA 共同工作。

6.10.1. 标准协议漫游

TBD...

6.10.2. 私有协议漫游

开启漫游功能后,协议栈在运行过程中会实时监测当前的 rssi。连续统计 n 个 beacon 帧的 rssi 进行累加平均,降低因信号抖动产生的影响。当检测 rssi 低于设定的漫游门限时,开始检查 AP 列表中是否存在更合适的 AP。

如果找到更合适的 AP 则向当前 AP 发起漫游请求,当前 AP 接收漫游请求后记录 STA 进入漫游状态,开始为 sta 的缓存数据,并回复 response 给 STA。

STA 接收到 response 后开始向新的 AP 发起连接请求,如果连接失败,则会回连到旧 AP,继续和旧 AP 通信。如果连接成功,旧 AP 则会将缓存的数据发送给新 AP,由新 AP 发送给 sta,避免因漫游产生数据丢失。

6.10.2.1. ieee80211_wnb_roam_enable

ieee80211_wnb_roam_enable API 用于初始化协议栈漫游功能。

- 参数 ifidx [[in](#)]: 接口索引。
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.10.2.2. ieee80211_conf_set_roaming

ieee80211_conf_set_roaming API 用于设置漫游功能开关。

- 参数 ifidx [[in](#)]: 接口索引。
- 参数 roaming [[in](#)]: 0: 关闭漫游, 1: 开启漫游
- 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

6.10.2.3. ieee80211_conf_set_roam_config

ieee80211_conf_set_roam_config API 用于设置漫游功能参数。

- 参数 ifidx [[in](#)]: 接口索引。

-
- 参数 `roam_rssi_th` [in]: 触发漫游的 rssi 门限
 - 参数 `roam_rssi_diff` [in]: 设置新旧 AP 的 rssi 差值, 大于此差值的新 AP 才能被选中, 避免频繁反复漫游切换。
 - 参数 `roam_rssi_int` [in]: 设置连续监测多少个 beacon 的 rssi 进行累加平均。
 - 返回值 : 表示是否成功, 0: 成功, 非 0: 失败。

7. 示例代码

7.1. 开启 AP 热点

AP 模式初始化后，启动 AP 模式通常只需要设置以下几个参数：

```
char psk[32];  
//设置 SSID 参数  
ieee80211_conf_set_ssid(WIFI_MODE_AP, "TEST_AP");  
//设置安全加密模式  
ieee80211_conf_set_keymgmt(WIFI_MODE_AP, WPA_KEY_MGMT_PSK);  
//根据密码和 SSID 计算 psk.  
//计算 psk 比较耗时，通常是计算后保存，避免下次开机再计算  
wpa_passphrase("TEST_AP", "12345678", psk);  
//设置 psk  
ieee80211_conf_set_psk(WIFI_MODE_AP, "TEST_AP");  
//设置 beacon 周期，默认是 100ms  
ieee80211_conf_set_beacon_int(WIFI_MODE_AP, 100);  
//设置 AP 的工作 channel  
ieee80211_conf_set_channel(WIFI_MODE_AP, 1);  
//启动 AP 接口  
ieee80211_iface_start(WIFI_MODE_AP);  
////////////////////////////////////  
//断开指定的 sta  
ieee80211_disassoc(WIFI_MODE_AP, sta_mac);  
//断开所有的 sta  
ieee80211_disassoc_all(WIFI_MODE_AP);
```

7.2. 开启 STA 连接 AP

STA 模式初始化后，启动 STA 连接指定的 AP 通常只需要设置以下几个参数：

```
char psk[32];  
//设置 SSID 参数  
ieee80211_conf_set_ssid(WIFI_MODE_STA, "TEST_AP");  
//设置安全加密模式  
ieee80211_conf_set_keymgmt(WIFI_MODE_STA, WPA_KEY_MGMT_PSK);  
//根据密码和 SSID 计算 psk.  
//计算 psk 比较耗时，通常是计算后保存，避免下次开机再计算  
wpa_passphrase("TEST_AP", "12345678", psk);  
//设置 psk  
ieee80211_conf_set_psk(WIFI_MODE_STA, psk);  
//启动 STA 接口  
ieee80211_iface_start(WIFI_MODE_STA);
```